

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN PHENIKAA



TÊN HỌC PHẦN: NHẬP MÔN TRÍ TUỆ NHÂN TẠO

**TÊN ĐỀ TÀI: CÀI ĐẶT PERCEPTRON VÀ MẠNG
NƠ-RON NHIỀU LỚP (MLP) TỪ ĐẦU BẰNG
NUMPY**

Giảng viên hướng dẫn: ThS. Phan Thị Hoài

Sinh viên thực hiện:

Ngô Quang Thiện - 24102651

Nguyễn Hưng Vũ - 24102769

Khoá: K18 - ICT3

Lớp tín chỉ: CSE703041-2-3-25(N02)

Chương trình đào tạo: Khoa Học Dữ Liệu và Trí Tuệ Nhân Tạo

Hà Nội, tháng 7 năm 2026

MỤC LỤC

1	MỤC TIÊU	3
1.1	Giới thiệu đề tài	3
1.2	Mục tiêu đề tài	3
1.3	Công cụ và môi trường thực hiện	3
2	CƠ SỞ LÝ THUYẾT	4
2.1	Quy ước ký hiệu	4
2.2	Mạng Perceptron một lớp và Giới hạn phân tách tuyến tính	4
2.2.1	Kiến trúc Perceptron	4
2.2.2	Giới hạn phân tách tuyến tính (Bài toán XOR)	6
2.3	Các hàm kích hoạt (Activation Function)	6
2.3.1	Hàm Sigmoid	6
2.3.2	Hàm ReLU	6
2.3.3	Hàm Softmax	7
2.4	Lan truyền thuận (Forward Propagation)	7
2.5	Hàm mất mát (Loss Function)	7
2.6	Lan truyền ngược (Backpropagation)	8
2.7	Các thuật toán tối ưu (Optimization Algorithms)	9
2.8	Regularization L2	9
3	KẾT QUẢ THỰC NGHIỆM	10
3.1	Cài đặt Perceptron đơn giản và bài toán XOR	10
3.2	Đánh giá kết quả cài đặt thuật toán MLP	10
3.3	Đánh giá trên bài toán XOR	11
3.4	Đánh giá trên tập dữ liệu Iris	12
4	THỰC NGHIỆM MỞ RỘNG	14
4.1	Đánh giá hiệu quả của thuật toán tối ưu SGD với Momentum	14
4.2	Ảnh hưởng của số lượng lớp ẩn (Hidden Layers)	14
4.3	Đánh giá hiệu quả của L2 Regularization	15
5	DỮ LIỆU VÀ MÔI TRƯỜNG XỬ LÝ	17
5.1	Tập dữ liệu	17
5.2	Môi trường và thư viện	17
6	KẾT LUẬN	18
6.1	Những kết quả đã đạt được	18
6.2	Những mặt còn hạn chế	18
6.3	Hướng phát triển trong tương lai	18

DANH MỤC HÌNH ẢNH

3.1	Perceptron trên bài toán XOR (Không thể phân tách)	10
3.2	Đồ thị các Hàm Kích Hoạt và Đạo hàm tương ứng	11
3.3	Tốc độ hội tụ và ranh giới quyết định của MLP trên bài toán XOR	12
3.4	Loss của MLP trên tập dữ liệu Iris (ReLU)	13
4.1	So sánh tốc độ hội tụ: SGD vs SGD+Momentum trên Iris	14
4.2	Ảnh hưởng của số lượng lớp ẩn đến quá trình huấn luyện	15
4.3	Đánh giá hiệu quả chống Overfitting của L2 Regularization	16

TÀI NGUYÊN MÃ NGUỒN (SOURCE CODE)

- Link Google Colab: <https://colab.research.google.com/drive/13pt17R3-004uGGPBXaLPsomusp=sharing>
- Link GitHub: https://github.com/thyelmot/BAO_CAO_NHAP_MON_TRI_TUE_NHAN_TAO

Chương 1

MỤC TIÊU

1.1 Giới thiệu đề tài

Trong những năm gần đây, Trí tuệ nhân tạo (Artificial Intelligence - AI) đã trở thành một trong những lĩnh vực phát triển mạnh mẽ nhất của khoa học máy tính. Một trong những nền tảng quan trọng của AI hiện đại là mạng nơ-ron nhân tạo (Artificial Neural Network - ANN), mô phỏng cách hoạt động của hệ thần kinh sinh học nhằm học và đưa ra quyết định từ dữ liệu.

Trong báo cáo này, nhóm tiến hành xây dựng mô hình Perceptron và Mạng nơ-ron nhiều lớp (Multi-Layer Perceptron - MLP) hoàn toàn bằng thư viện NumPy, không sử dụng các framework học sâu như TensorFlow hay PyTorch. Việc cài đặt từ đầu giúp hiểu rõ cơ chế lan truyền thuận (Forward Propagation), lan truyền ngược (Backpropagation) và quá trình tối ưu hóa tham số.

1.2 Mục tiêu đề tài

Đề tài nhằm tìm hiểu nguyên lý hoạt động của mạng nơ-ron nhân tạo thông qua việc tự cài đặt Perceptron và mạng nơ-ron nhiều lớp (MLP) hoàn toàn bằng thư viện NumPy, không sử dụng các framework học sâu như TensorFlow hay PyTorch. Bên cạnh đó, đề tài giúp làm rõ quá trình lan truyền thuận (forward propagation), lan truyền ngược (backpropagation) và cơ chế cập nhật trọng số trong quá trình huấn luyện. Cuối cùng, mô hình được đánh giá trên hai bài toán XOR và Iris, đồng thời so sánh hiệu quả của các phương pháp tối ưu SGD, SGD with Momentum và ảnh hưởng của regularization L2 đến khả năng hội tụ và độ chính xác của mô hình.

1.3 Công cụ và môi trường thực hiện

Đề tài được thực hiện bằng ngôn ngữ lập trình Python nhờ cú pháp đơn giản và hệ sinh thái phong phú trong lĩnh vực khoa học dữ liệu và học máy. Quá trình cài đặt mô hình sử dụng NumPy để thực hiện các phép toán ma trận và xây dựng toàn bộ thuật toán Perceptron, MLP, lan truyền thuận (Forward Propagation) và lan truyền ngược (Backpropagation) từ đầu mà không sử dụng các framework học sâu như TensorFlow hay PyTorch. Ngoài ra, Matplotlib được sử dụng để trực quan hóa quá trình huấn luyện thông qua các đồ thị như hàm mất mát (Loss) và độ chính xác (Accuracy), trong khi Scikit-learn được dùng để tải tập dữ liệu Iris, hỗ trợ chia tập huấn luyện/kiểm thử và đánh giá kết quả mô hình. Toàn bộ mã nguồn được phát triển và thử nghiệm trên môi trường Visual Studio Code giúp thuận tiện cho việc lập trình, kiểm thử và phân tích kết quả thực nghiệm.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Quy ước ký hiệu

Để đảm bảo tính thống nhất trong toàn bộ báo cáo, các ký hiệu toán học được sử dụng như Bảng 2.1. Theo quy ước phổ biến trong học sâu (Deep Learning), chữ hoa biểu diễn ma trận, còn chữ thường biểu diễn vector hoặc đại lượng vô hướng tùy theo ngữ cảnh.

Bảng 2.1: Các ký hiệu sử dụng trong báo cáo

Ký hiệu	Ý nghĩa
X	Ma trận dữ liệu đầu vào gồm nhiều mẫu
x	Vector đặc trưng của một mẫu dữ liệu
$W^{(l)}$	Ma trận trọng số của lớp thứ l
w	Vector trọng số của một nơ-ron
$b^{(l)}$	Vector bias của lớp thứ l
z	Giá trị trước hàm kích hoạt của một nơ-ron
$Z^{(l)}$	Ma trận giá trị trước kích hoạt của lớp thứ l
a	Đầu ra của một nơ-ron sau hàm kích hoạt
$A^{(l)}$	Ma trận đầu ra của lớp thứ l sau hàm kích hoạt
$\hat{y}$	Giá trị dự đoán của mô hình ($\hat{y} = A^{(L)}$)
y	Nhãn thực tế
$\delta^{(l)}$	Sai số lan truyền ngược của lớp thứ l
η	Tốc độ học (Learning Rate)
β	Hệ số Momentum
λ	Hệ số L2 Regularization
$\mathcal{L}$	Hàm mất mát (Loss Function)
N	Số lượng mẫu dữ liệu
C	Số lượng lớp cần phân loại

2.2 Mạng Perceptron một lớp và Giới hạn phân tách tuyến tính

2.2.1 Kiến trúc Perceptron

Perceptron là mô hình mạng nơ-ron nhân tạo đơn giản nhất. Mô hình này mô phỏng hoạt động của một nơ-ron sinh học bằng cách kết hợp các đầu vào thông qua trọng số và hàm kích hoạt để đưa ra kết quả phân loại. Perceptron có nhiệm vụ tìm một siêu phẳng (hyperplane) nhằm phân tách các lớp dữ liệu và được sử dụng trong các bài toán phân loại nhị phân. Tuy nhiên, mô hình chỉ hoạt động hiệu quả với dữ liệu phân tách tuyến tính và không thể giải quyết các

bài toán phi tuyến như XOR.

Một Perceptron gồm ba thành phần chính: lớp đầu vào (Input Layer), trọng số và bias (Weights & Bias), và hàm kích hoạt (Activation Function). Lớp đầu vào tiếp nhận các đặc trưng của dữ liệu, mỗi đặc trưng được gán một trọng số w_i thể hiện mức độ ảnh hưởng đến kết quả. Giá trị bias b giúp điều chỉnh ngưỡng kích hoạt, tăng tính linh hoạt của mô hình. Tổng có trọng số của các đầu vào sau đó được đưa qua hàm kích hoạt để tạo ra đầu ra cuối cùng, thường là 0 hoặc 1 trong bài toán phân loại nhị phân.

Nguyên lý hoạt động của Perceptron gồm ba bước: (1) nhân các đặc trưng đầu vào với trọng số tương ứng, (2) cộng các giá trị này với bias để tạo tổng có trọng số, và (3) đưa kết quả qua hàm kích hoạt để xác định đầu ra. Trong quá trình huấn luyện, mô hình so sánh kết quả dự đoán với nhãn thực tế và cập nhật trọng số cùng bias khi dự đoán sai. Quá trình này được lặp lại nhiều lần cho đến khi mô hình hội tụ hoặc đạt số vòng lặp tối đa.

Công thức toán học:

Giả sử vector đầu vào của một mẫu dữ liệu là:

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T \quad (2.1)$$

và vector trọng số tương ứng là:

$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T \quad (2.2)$$

Giá trị đầu vào của nơ-ron được tính theo công thức:

$$z = \sum_{i=1}^n w_i x_i + b \quad (2.3)$$

hay viết dưới dạng ma trận:

$$z = \mathbf{w}^T \mathbf{x} + b \quad (2.4)$$

Sau đó, đầu ra của Perceptron được xác định thông qua hàm kích hoạt:

$$\hat{y} = f(z) \quad (2.5)$$

Trong đó, hàm kích hoạt bước (Step Function) thường được sử dụng:

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases} \quad (2.6)$$

Khi mô hình dự đoán sai, các trọng số và bias được cập nhật theo quy tắc học Perceptron:

$$w_i = w_i + \eta(y - \hat{y})x_i \quad (2.7)$$

$$b = b + \eta(y - \hat{y}) \quad (2.8)$$

Trong đó:

- y : nhãn thực tế.
- $\hat{y}$: nhãn dự đoán.
- η : tốc độ học (learning rate).
- w_i : trọng số của đặc trưng thứ i .
- b : hệ số bias.

2.2.2 Giới hạn phân tách tuyến tính (Bài toán XOR)

Perceptron là mô hình phân loại tuyến tính, chỉ có khả năng giải quyết các bài toán có dữ liệu phân tách tuyến tính (Linearly Separable), tức là có thể được phân chia hoàn toàn bằng một đường thẳng, mặt phẳng hoặc siêu phẳng. Trong quá trình huấn luyện, Perceptron điều chỉnh trọng số và bias để tìm ranh giới phân tách giữa các lớp dữ liệu. Nếu dữ liệu phân tách tuyến tính, mô hình sẽ hội tụ và phân loại chính xác. Ngược lại, với các bài toán không phân tách tuyến tính, Perceptron không thể tìm được nghiệm phù hợp và sẽ không hội tụ.

Bài toán XOR (Exclusive OR) là một ví dụ kinh điển minh họa cho giới hạn của Perceptron. Hàm XOR cho kết quả bằng 1 khi hai đầu vào khác nhau và bằng 0 khi hai đầu vào giống nhau.

Bảng chân trị của phép toán XOR được thể hiện như sau:

x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Các điểm dữ liệu của XOR được phân bố theo hai đường chéo đối diện, nên không thể phân tách hoàn toàn bằng một đường thẳng. Do đó, XOR là một bài toán không phân tách tuyến tính (Non-linearly Separable) và là ví dụ kinh điển minh họa giới hạn của Perceptron một lớp.

2.3 Các hàm kích hoạt (Activation Function)

Hàm kích hoạt (Activation Function) có nhiệm vụ chuyển đổi tổng có trọng số của đầu vào thành đầu ra của nơ-ron, đồng thời tạo tính phi tuyến cho mạng nơ-ron. Nhờ đó, mô hình có thể học các mối quan hệ phức tạp và giải quyết các bài toán phi tuyến như XOR. Việc lựa chọn hàm kích hoạt ảnh hưởng trực tiếp đến khả năng học và hiệu quả của mô hình. Trong đề tài này, các hàm được sử dụng gồm Sigmoid, ReLU và Softmax.

2.3.1 Hàm Sigmoid

Sigmoid là hàm kích hoạt biến đổi giá trị đầu vào về khoảng $(0, 1)$, thường được sử dụng trong bài toán phân loại nhị phân.

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.9)$$

Đạo hàm của Sigmoid được tính theo công thức:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \quad (2.10)$$

2.3.2 Hàm ReLU

ReLU là hàm kích hoạt phổ biến trong các Hidden Layer nhờ khả năng tính toán nhanh và giảm hiện tượng mất gradient.

$$f(z) = \max(0, z) \quad (2.11)$$

Đạo hàm của ReLU:

$$f'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases} \quad (2.12)$$

2.3.3 Hàm Softmax

Softmax được sử dụng ở lớp đầu ra (L) của bài toán phân loại nhiều lớp, chuyển vector đầu vào thành phân phối xác suất có tổng bằng 1. Nhất quán với quy ước ký hiệu trong Bảng 2.1, đầu ra của lớp Softmax được ký hiệu là $A^{(L)}$:

$$\hat{y}_c = A_c^{(L)} = \frac{e^{z_c}}{\sum_{j=1}^C e^{z_j}} \quad (2.13)$$

hoặc viết gọn dưới dạng ma trận:

$$A^{(L)} = \text{Softmax}(Z^{(L)}) \quad (2.14)$$

2.4 Lan truyền thuận (Forward Propagation)

Lan truyền thuận (Forward Propagation) là quá trình truyền dữ liệu từ Input Layer qua các Hidden Layer đến Output Layer để tạo ra kết quả dự đoán của mô hình. Ở mỗi nơ-ron, dữ liệu được nhân với trọng số (Weight), cộng Bias, sau đó đi qua hàm kích hoạt trước khi truyền sang lớp tiếp theo.

Nguyên lý hoạt động:

Quá trình lan truyền thuận gồm các bước:

- **Bước 1:** Nhận vector đầu vào $\mathbf{x} = [x_1, x_2, \dots, x_n]$
- **Bước 2:** Tính tổng có trọng số $z = \mathbf{w}^T \mathbf{x} + b$
- **Bước 3:** Áp dụng hàm kích hoạt $a = f(z)$ (Trong đó, $f(\cdot)$ có thể là Sigmoid, ReLU hoặc Softmax).
- **Bước 4:** Đầu ra của lớp hiện tại được truyền sang lớp tiếp theo và lặp lại cho đến Output Layer để tạo ra kết quả dự đoán $\hat{y}$.

Công thức tổng quát:

Đối với lớp thứ l của mạng MLP, quá trình lan truyền thuận được mô tả bởi:

$$Z^{(l)} = W^{(l)} A^{(l-1)} + b^{(l)} \quad (2.15)$$

$$A^{(l)} = f(Z^{(l)}) \quad (2.16)$$

Lưu ý: Trong phần lý thuyết, các công thức được trình bày theo quy ước vector cột. Khi hiện thực bằng NumPy, mỗi hàng của ma trận tương ứng với một mẫu dữ liệu, do đó phép nhân được triển khai dưới dạng $Z = AW + b$. Hai cách biểu diễn là tương đương và chỉ khác nhau ở quy ước lưu trữ dữ liệu.

Vai trò của Forward Propagation:

Forward Propagation là bước tạo ra kết quả dự đoán từ dữ liệu đầu vào và là cơ sở để tính hàm mất mát (Loss Function). Giá trị Loss sau đó được sử dụng trong Backpropagation để tính gradient và cập nhật trọng số của mạng. Hai quá trình này phối hợp trong mỗi vòng huấn luyện, giúp mô hình dần học được quy luật của dữ liệu và nâng cao độ chính xác dự đoán.

2.5 Hàm mất mát (Loss Function)

Hàm mất mát (Loss Function) là hàm toán học dùng để đo lường sai lệch giữa giá trị dự đoán và giá trị thực tế. Giá trị Loss càng nhỏ thì mô hình dự đoán càng chính xác.

Mean Squared Error (MSE): Dùng cho bài toán hồi quy (tính trên N mẫu dữ liệu).

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2.17)$$

Cross-Entropy Loss: Dùng cho bài toán phân loại nhiều lớp (tính trên N mẫu dữ liệu, mỗi mẫu có C lớp).

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log(\hat{y}_{ic}) \quad (2.18)$$

Vai trò:

Trong mỗi vòng huấn luyện, mạng nơ-ron thực hiện theo quy trình: Forward Propagation $\rightarrow$ Loss Function $\rightarrow$ Backpropagation $\rightarrow$ Cập nhật trọng số.

2.6 Lan truyền ngược (Backpropagation)

Lan truyền ngược (Backpropagation) là thuật toán dùng để huấn luyện mạng nơ-ron nhiều lớp (MLP). Sau khi Forward Propagation tạo ra kết quả dự đoán và tính Loss, Backpropagation sẽ lan truyền sai số từ Output Layer về Hidden Layer, tính gradient của từng trọng số và bias bằng quy tắc đạo hàm dây chuyền (Chain Rule).

Quy tắc đạo hàm dây chuyền (Chain Rule):

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} \quad (2.19)$$

Tính Gradient:

Đạo hàm của hàm mất mát theo đầu vào của lớp cuối cùng (kết hợp Softmax và Cross-Entropy) được tính gọn bằng:

$$\delta^{(L)} = A^{(L)} - Y \quad (2.20)$$

Sai số tại lớp ẩn được tính theo:

$$\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot f'(Z^{(l)}) \quad (2.21)$$

Gradient của trọng số và bias:

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \delta^{(l)} (A^{(l-1)})^T \quad (2.22)$$

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \delta^{(l)} \quad (2.23)$$

Lưu ý: Công thức (2.23) áp dụng cho trường hợp lan truyền ngược của một mẫu dữ liệu (single sample). Khi huấn luyện theo mini-batch gồm N mẫu, gradient của bias được tính bằng tổng hoặc trung bình các vector sai số của toàn bộ batch:

$$\frac{\partial \mathcal{L}}{\partial b^{(l)}} = \frac{1}{N} \sum_{i=1}^N \delta_i^{(l)} \quad (2.24)$$

Cập nhật trọng số:

$$W^{(l)} = W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (2.25)$$

$$b^{(l)} = b^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}} \quad (2.26)$$

2.7 Các thuật toán tối ưu (Optimization Algorithms)

Thuật toán tối ưu (Optimization Algorithm) là phương pháp cập nhật trọng số và bias dựa trên gradient nhằm giảm hàm mất mát. Để tổng quát cho mạng nhiều lớp, ta dùng ký hiệu (l) chỉ lớp thứ l .

Gradient Descent (GD):

$$W^{(l)} = W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (2.27)$$

$$b^{(l)} = b^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}} \quad (2.28)$$

Stochastic Gradient Descent (SGD):

$$W^{(l)} = W^{(l)} - \eta \frac{\partial \mathcal{L}_i}{\partial W^{(l)}} \quad (2.29)$$

$$b^{(l)} = b^{(l)} - \eta \frac{\partial \mathcal{L}_i}{\partial b^{(l)}} \quad (2.30)$$

SGD with Momentum:

$$v_t^{W^{(l)}} = \beta v_{t-1}^{W^{(l)}} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}} \quad (2.31)$$

$$W^{(l)} = W^{(l)} + v_t^{W^{(l)}} \quad (2.32)$$

$$v_t^{b^{(l)}} = \beta v_{t-1}^{b^{(l)}} - \eta \frac{\partial \mathcal{L}}{\partial b^{(l)}} \quad (2.33)$$

$$b^{(l)} = b^{(l)} + v_t^{b^{(l)}} \quad (2.34)$$

Trong đó, β là hệ số Momentum (thường khoảng 0.9).

2.8 Regularization L2

L2 Regularization (Weight Decay) giúp giảm hiện tượng overfitting bằng cách thêm một thành phần phạt vào hàm mất mát.

$$\mathcal{L}' = \mathcal{L} + \frac{\lambda}{2N} \sum_{l=1}^L \|W^{(l)}\|_F^2 \quad (2.35)$$

Gradient và cập nhật trọng số sau khi thêm L2 (chỉ áp dụng cho trọng số, không áp dụng cho bias):

$$\frac{\partial \mathcal{L}'}{\partial W^{(l)}} = \frac{\partial \mathcal{L}}{\partial W^{(l)}} + \frac{\lambda}{N} W^{(l)} \quad (2.36)$$

$$W^{(l)} = W^{(l)} - \eta \left(\frac{\partial \mathcal{L}}{\partial W^{(l)}} + \frac{\lambda}{N} W^{(l)} \right) \quad (2.37)$$

Chương 3

KẾT QUẢ THỰC NGHIỆM

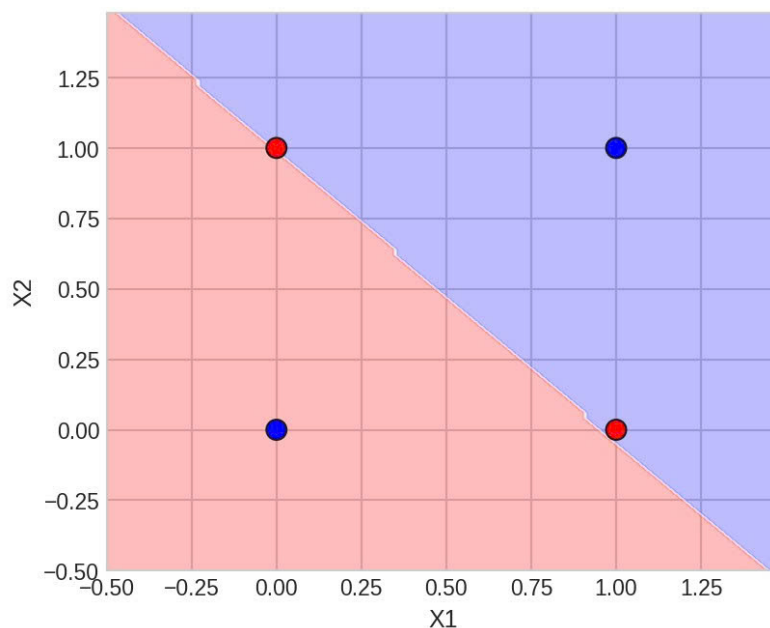
3.1 Cài đặt Perceptron đơn giản và bài toán XOR

Nhóm tiến hành huấn luyện Perceptron một lớp với Step Function trên tập dữ liệu XOR gồm bốn mẫu $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$ và $(1, 1) \rightarrow 0$.

Kết quả thực nghiệm cho thấy mô hình không thể hội tụ sau quá trình huấn luyện, giá trị Loss không giảm về 0 và độ chính xác chỉ đạt khoảng 75% (phân loại sai điểm $(1, 1)$). Các dự đoán vẫn còn sai do Perceptron không tìm được ranh giới phân tách phù hợp.

Biểu đồ Decision Boundary cho thấy Perceptron chỉ tạo được một ranh giới quyết định tuyến tính, trong khi dữ liệu XOR có phân bố không phân tách tuyến tính, nên không thể phân loại chính xác tất cả các mẫu.

Kết quả này hoàn toàn phù hợp với lý thuyết, khẳng định Perceptron chỉ giải quyết được các bài toán phân tách tuyến tính. Để xử lý các bài toán phi tuyến như XOR, cần sử dụng Mạng nơ-ron nhiều lớp (MLP) kết hợp với các hàm kích hoạt phi tuyến như Sigmoid hoặc ReLU.



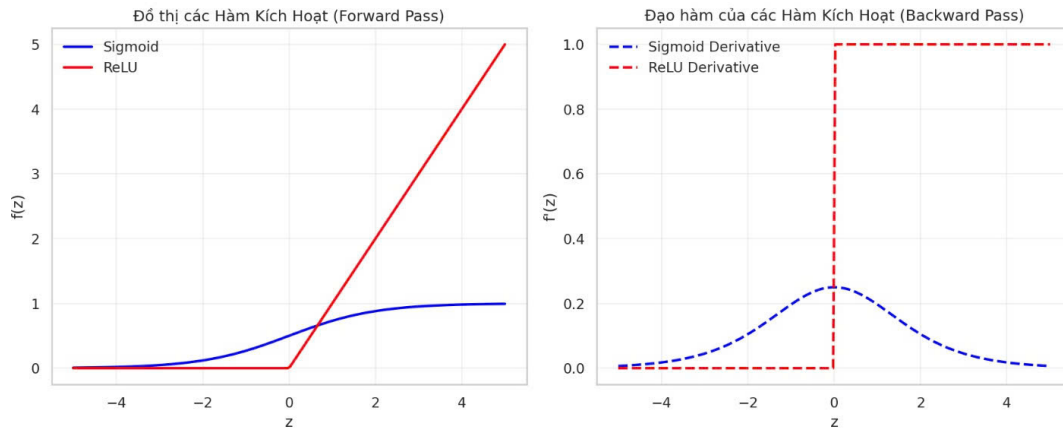
Hình 3.1: Perceptron trên bài toán XOR (Không thể phân tách)

3.2 Đánh giá kết quả cài đặt thuật toán MLP

Nhóm đã cài đặt thành công Mạng nơ-ron nhiều lớp (MLP) hoàn toàn bằng NumPy, không sử dụng các framework học sâu như TensorFlow hay PyTorch. Mô hình được thiết kế linh hoạt thông qua tham số `layer_sizes`, cho phép dễ dàng thay đổi số lượng lớp ẩn và số nơ-ron của từng lớp.

Quá trình Forward Propagation được triển khai bằng phép nhân ma trận, hỗ trợ các hàm kích hoạt Sigmoid và ReLU, đồng thời tự động lựa chọn Sigmoid hoặc Softmax ở lớp đầu ra kết hợp với Cross-Entropy Loss phù hợp với từng bài toán. Các giá trị trung gian được lưu lại để phục vụ cho quá trình lan truyền ngược.

Đối với Backward Propagation, nhóm đã cài đặt thành công thuật toán dựa trên quy tắc đạo hàm dây chuyền (Chain Rule) để tính gradient của trọng số và bias. Quá trình lan truyền sai số giữa các lớp được thực hiện chính xác bằng các phép toán ma trận NumPy. Kết quả thực nghiệm cho thấy thuật toán hoạt động ổn định, các trọng số được cập nhật chính xác thông qua Gradient Descent.

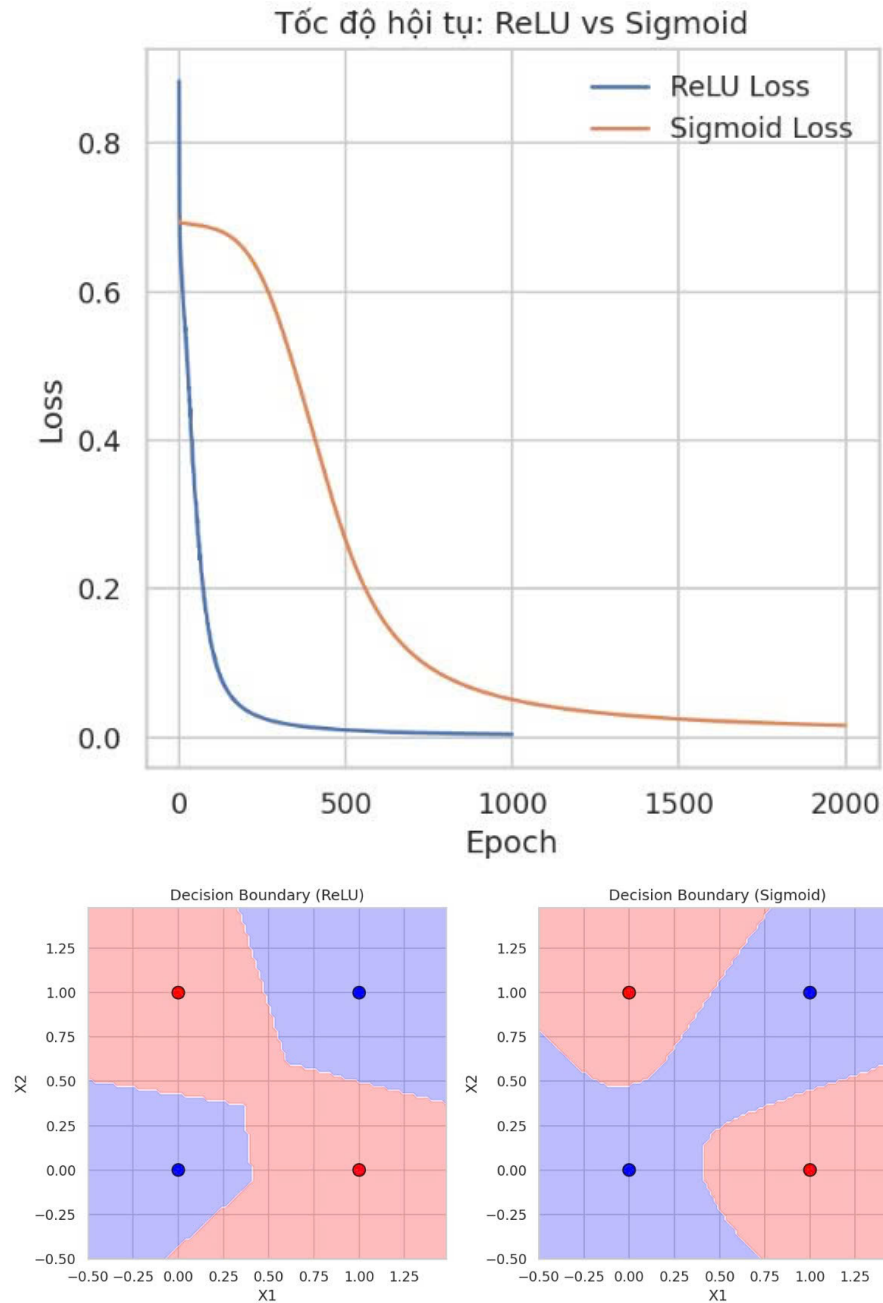


Hình 3.2: Đồ thị các Hàm Kích Hoạt và Đạo hàm tương ứng

3.3 Đánh giá trên bài toán XOR

Đối với bài toán XOR, mạng MLP được cấu hình theo kiến trúc 2 Input – 4 Hidden – 1 Output và sử dụng hai hàm kích hoạt Sigmoid và ReLU để so sánh hiệu quả huấn luyện.

Kết quả thực nghiệm cho thấy mô hình MLP hội tụ thành công và phân loại chính xác 100% các mẫu dữ liệu XOR, khắc phục hoàn toàn hạn chế của Perceptron một lớp. Biểu đồ Decision Boundary cho thấy mạng đã học được ranh giới quyết định phi tuyến. Đồng thời, đồ thị Loss theo Epoch cho thấy ReLU hội tụ nhanh hơn Sigmoid.

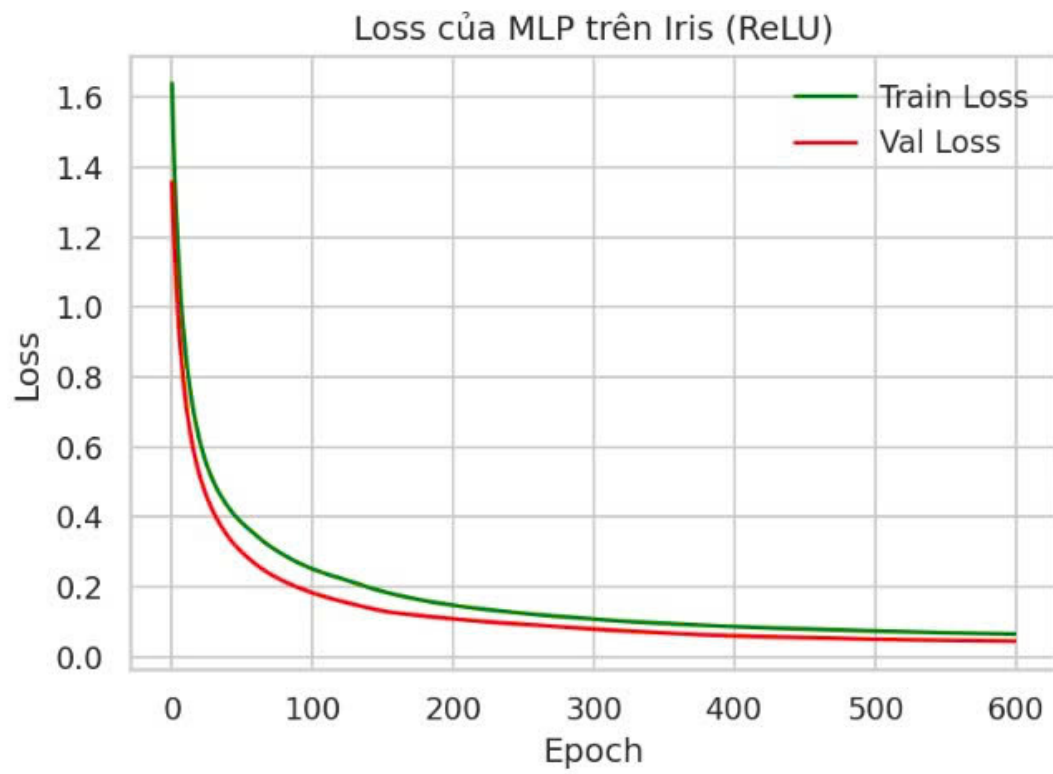


Hình 3.3: Tốc độ hội tụ và ranh giới quyết định của MLP trên bài toán XOR

3.4 Đánh giá trên tập dữ liệu Iris

Trước khi huấn luyện, dữ liệu Iris được chuẩn hóa bằng StandardScaler, chuyển đổi nhãn sang One-hot Encoding và chia theo tỷ lệ 80% dữ liệu huấn luyện và 20% dữ liệu kiểm tra. Mô hình được cấu hình theo kiến trúc 4 Input – 8 Hidden – 3 Output, sử dụng ReLU cho lớp ẩn, Softmax cho lớp đầu ra và Learning Rate = 0.1.

Kết quả huấn luyện cho thấy Train Loss và Validation Loss giảm đều và hội tụ sau khoảng 600 epochs. Khi đánh giá trên tập kiểm tra, mô hình đạt độ chính xác từ 96% đến 100%.



Hình 3.4: Loss của MLP trên tập dữ liệu Iris (ReLU)

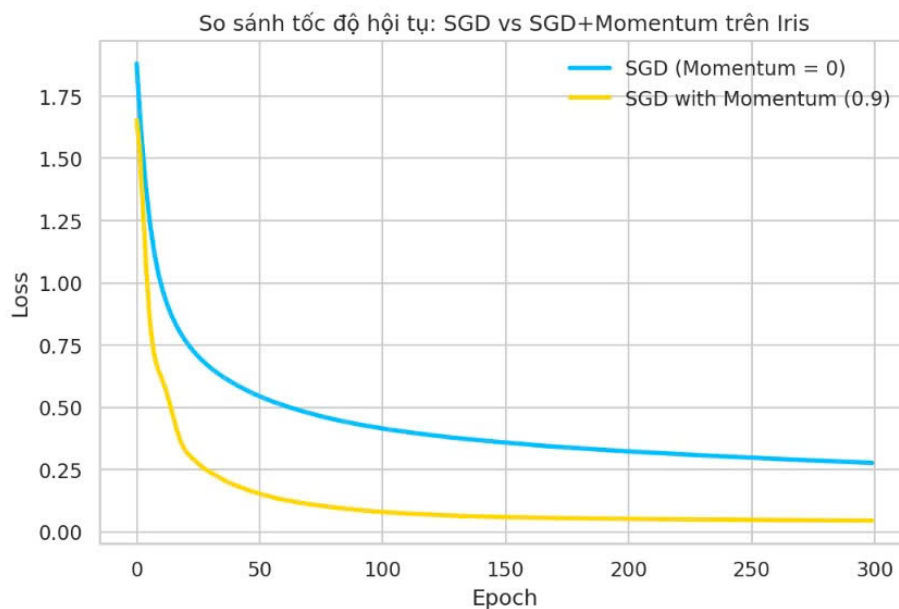
Chương 4

THỰC NGHIỆM MỞ RỘNG

4.1 Đánh giá hiệu quả của thuật toán tối ưu SGD với Momentum

Nhóm tiến hành huấn luyện hai mô hình MLP trên tập dữ liệu Iris, sử dụng lần lượt SGD và SGD with Momentum ($\beta = 0.9$) với cùng cấu hình và số epoch để đánh giá hiệu quả của thuật toán tối ưu.

Kết quả thực nghiệm cho thấy SGD with Momentum có tốc độ hội tụ vượt trội. Đường Loss giảm nhanh và đạt mức khoảng 0.05 chỉ sau gần 100 epochs, trong khi SGD vẫn còn ở mức khoảng 0.18 sau 300 epochs. Điều này cho thấy Momentum giúp mô hình tìm đến điểm tối ưu nhanh hơn.



Hình 4.1: So sánh tốc độ hội tụ: SGD vs SGD+Momentum trên Iris

4.2 Ảnh hưởng của số lượng lớp ẩn (Hidden Layers)

Nhóm tiến hành huấn luyện ba kiến trúc MLP trên tập dữ liệu Iris trong 500 epochs:

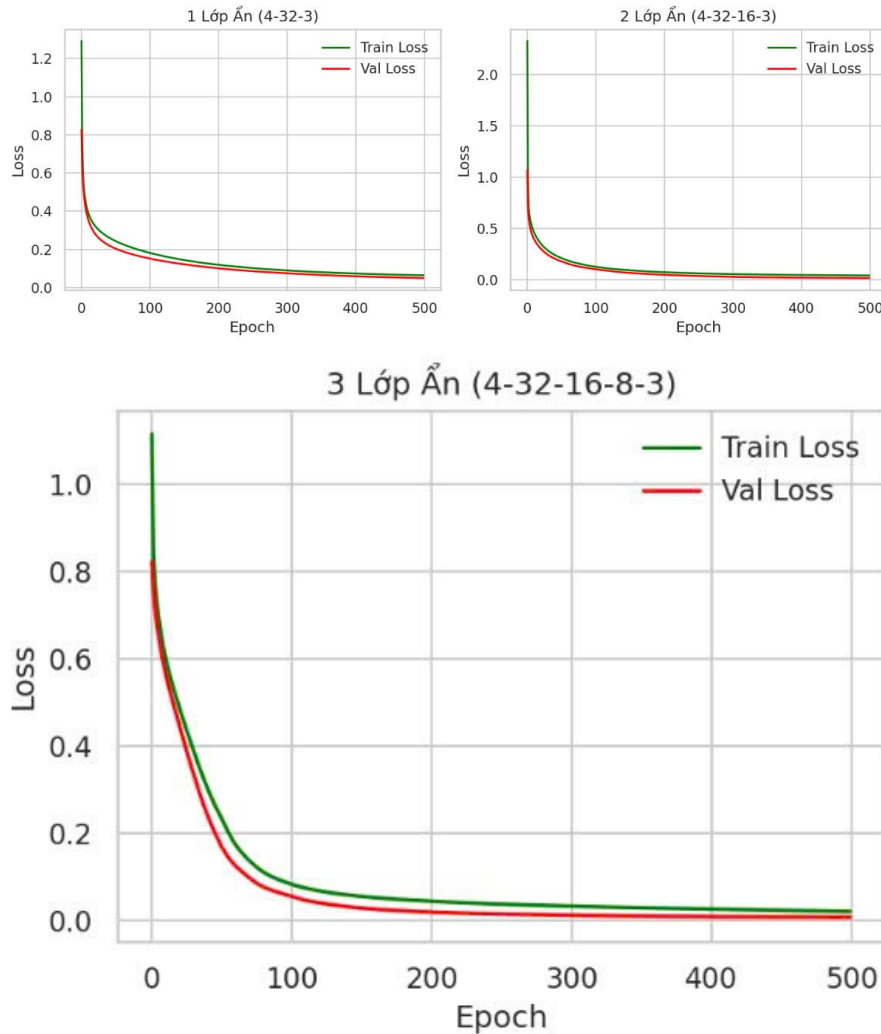
- 1 lớp ẩn: [4–32–3]
- 2 lớp ẩn: [4–32–16–3]
- 3 lớp ẩn: [4–32–16–8–3]

Kết quả thực nghiệm:

- **Mô hình 1 lớp ẩn:** Train Loss và Validation Loss giảm đều nhưng hội tụ chậm, đạt khoảng 0.06 sau gần 500 epochs.
- **Mô hình 2 lớp ẩn:** Hội tụ nhanh nhất, cả Train Loss và Validation Loss giảm mạnh và đạt khoảng 0.04 chỉ sau 150–200 epochs.

- **Mô hình 3 lớp ẩn:** Loss tiếp tục giảm đến khoảng 0.02, tuy nhiên tốc độ và độ chính xác chỉ cải thiện rất ít so với mô hình 2 lớp ẩn, trong khi số lượng tham số tăng đáng kể.

Kiến trúc [4-32-16-3] là lựa chọn tối ưu cho bài toán phân loại Iris, giúp mô hình học nhanh, ổn định và có khả năng tổng quát hóa tốt.

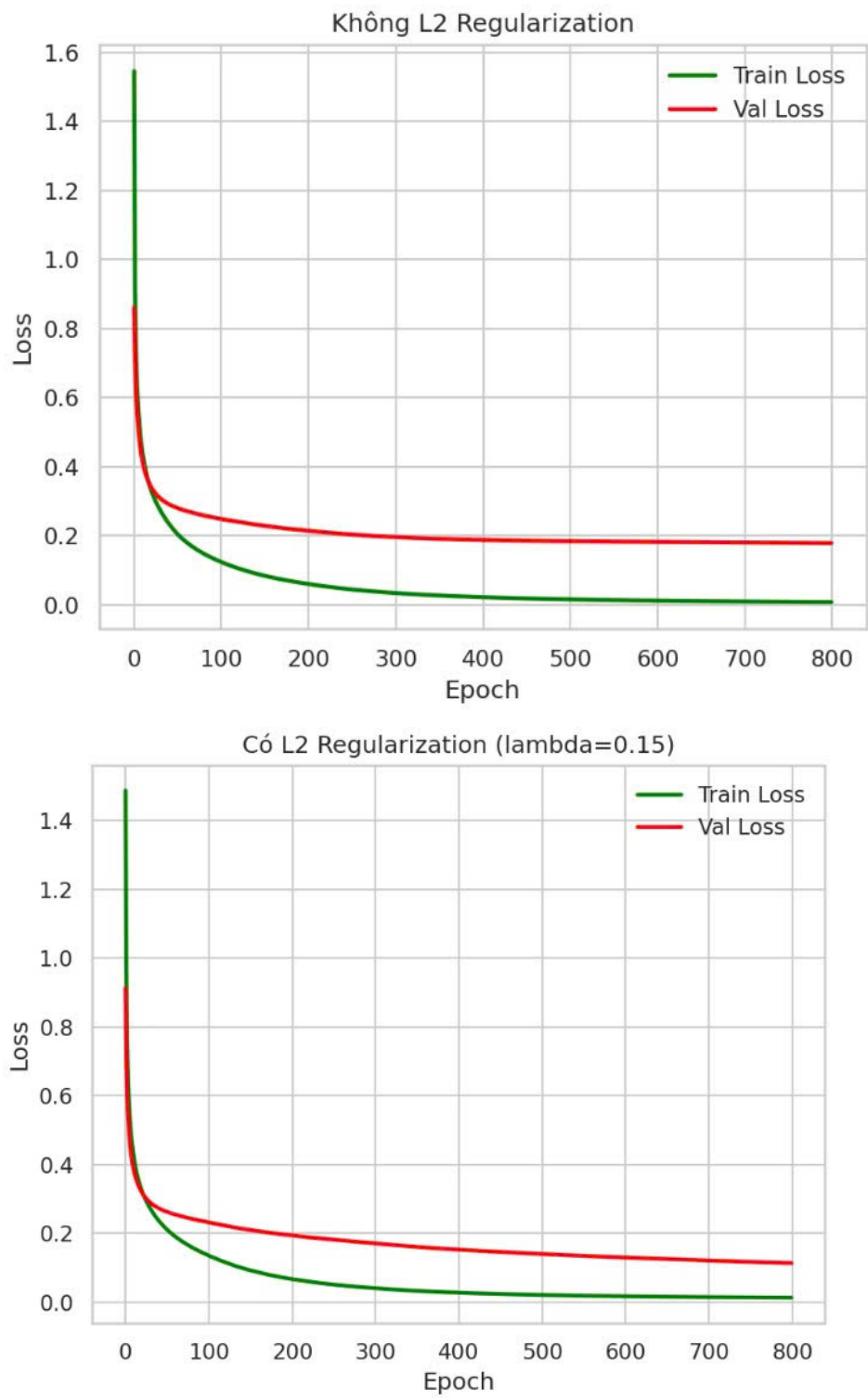


Hình 4.2: Ảnh hưởng của số lượng lớp ẩn đến quá trình huấn luyện

4.3 Đánh giá hiệu quả của L2 Regularization

Để đánh giá khả năng chống overfitting, nhóm sử dụng mô hình MLP có cấu trúc lớn (4-64-32-3) và chỉ huấn luyện trên 30 mẫu dữ liệu trong 800 epochs. Thực nghiệm được so sánh giữa hai trường hợp không sử dụng L2 ($\lambda = 0$) và có sử dụng L2 ($\lambda = 0.15$).

Kết quả thực nghiệm: Khi không sử dụng L2, Train Loss giảm rất nhanh về gần 0 trong khi Validation Loss dừng ở mức cao (khoảng 0.2), cho thấy mô hình bị overfitting. Ngược lại, khi áp dụng L2 Regularization, Train Loss giảm chậm hơn nhưng ổn định, đồng thời Validation Loss tiếp tục giảm xuống khoảng 0.1, khoảng cách giữa Train Loss và Validation Loss được thu hẹp đáng kể.



Hình 4.3: Đánh giá hiệu quả chống Overfitting của L2 Regularization

Chương 5

DỮ LIỆU VÀ MÔI TRƯỜNG XỬ LÝ

5.1 Tập dữ liệu

Đề tài sử dụng hai tập dữ liệu để kiểm thử và đánh giá mô hình:

- **Tập dữ liệu XOR:** Gồm 4 mẫu nhị phân $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$. Bộ dữ liệu này được sử dụng để kiểm chứng giới hạn của Perceptron và khả năng giải quyết bài toán phi tuyến của mạng MLP.
- **Tập dữ liệu Iris:** Bộ dữ liệu gồm 150 mẫu, thuộc 3 loài hoa Iris, mỗi mẫu có 4 đặc trưng. Dữ liệu được chuẩn hóa bằng StandardScaler, nhân được mã hóa One-Hot Encoding, sau đó chia thành tập huấn luyện (80%) và kiểm tra (20%).

5.2 Môi trường và thư viện

Các thực nghiệm được thực hiện trên Python 3.13 trong Google Colab (Visual Studio Code). Các thư viện sử dụng gồm:

- **NumPy:** Cài đặt toàn bộ mô hình Perceptron, MLP, Forward Propagation, Backpropagation, SGD, Momentum và L2 Regularization.
- **Matplotlib, Seaborn:** Trực quan hóa đồ thị Loss, Decision Boundary và Confusion Matrix.
- **Scikit-learn:** Hỗ trợ tải dữ liệu Iris, chuẩn hóa dữ liệu, chia tập huấn luyện/kiểm tra và đánh giá mô hình.

Chương 6

KẾT LUẬN

6.1 Những kết quả đã đạt được

Trong đề tài này, nhóm đã hoàn thành xuất sắc các mục tiêu đề ra ban đầu, bao gồm:

- **Cài đặt thành công mô hình từ đầu (scratch):** Xây dựng thành công mô hình Perceptron và Mạng nơ-ron nhiều lớp (MLP) hoàn toàn bằng thư viện toán học NumPy mà không phụ thuộc vào các framework học sâu có sẵn (như PyTorch hay TensorFlow).
- **Am hiểu sâu sắc cơ sở toán học:** Áp dụng chính xác các công thức toán học về Lan truyền thuận (Forward Propagation) và Lan truyền ngược (Backpropagation) sử dụng quy tắc đạo hàm dây chuyền (Chain Rule) trên hệ thống ma trận (Batch Matrix).
- **Phát triển các thuật toán tối ưu tiên tiến:** Không chỉ dừng lại ở thuật toán cập nhật cơ bản (SGD), nhóm đã tự cài đặt thành công thuật toán tối ưu có gia tốc (SGD with Momentum) giúp tăng tốc độ hội tụ, và thuật toán điều chuẩn (L2 Regularization / Weight Decay) giúp giảm thiểu mạnh mẽ hiện tượng Overfitting.
- **Thực nghiệm và trực quan hóa chi tiết:** Đánh giá toàn diện trên cả dữ liệu phi tuyến (XOR) và dữ liệu thực tế nhiều đặc trưng (Iris). Trực quan hóa và phân tích chi tiết các đồ thị hàm mất mát, ranh giới quyết định (Decision Boundary), qua đó minh chứng khoa học được ưu/nhược điểm của từng cấu hình mạng.

6.2 Những mặt còn hạn chế

Mặc dù đã đạt được nhiều kết quả tích cực, mô hình và đề tài vẫn còn một số hạn chế nhất định:

- **Hiệu suất tính toán:** Việc tính toán ma trận hoàn toàn bằng CPU thông qua NumPy sẽ gặp trở ngại về tốc độ nếu mở rộng sang các tập dữ liệu kích thước lớn (như xử lý ảnh, văn bản), do chưa tận dụng được sức mạnh tính toán song song của GPU (CUDA).
- **Thuật toán tối ưu:** Đề tài mới chỉ áp dụng SGD with Momentum. Trong thực tế, các kiến trúc Deep Learning hiện đại thường yêu cầu các thuật toán tối ưu tự thích ứng (Adaptive Learning Rate) như Adam hay RMSprop để hội tụ tốt hơn trên các không gian hao phí phức tạp.
- **Cơ chế dừng sớm (Early Stopping):** Quá trình huấn luyện hiện tại chạy cố định theo số vòng lặp (Epochs) được định trước. Nếu mô hình đã tìm được điểm tối ưu cục bộ sớm, việc tiếp tục tính toán gây lãng phí tài nguyên.

6.3 Hướng phát triển trong tương lai

Từ những hạn chế trên, nhóm đề xuất các hướng phát triển tiếp theo cho đề tài:

- **Tích hợp thuật toán Adam:** Cài đặt bổ sung thuật toán Adam (Adaptive Moment Estimation) để tự động điều chỉnh tốc độ học cho từng tham số mạng, giúp quá trình hội tụ nhanh, ổn định và ít phụ thuộc vào việc điều chỉnh siêu tham số thủ công.

- **Phát triển cơ chế Early Stopping:** Lập trình module theo dõi độ đo trên tập Validation, tự động ngắt quá trình huấn luyện khi Validation Loss có dấu hiệu tăng trở lại (dấu hiệu của Overfitting), giúp tiết kiệm thời gian và tài nguyên.
- **Mở rộng dữ liệu và kiến trúc:** Thử nghiệm mô hình tự xây dựng trên các tập dữ liệu lớn và phức tạp hơn như nhận dạng chữ số viết tay (MNIST) hoặc dữ liệu ảnh màu (CIFAR-10) để kiểm tra tính bền vững của mã nguồn.
- **Đóng gói thành thư viện độc lập:** Tái cấu trúc mã nguồn theo hướng đối tượng (OOP) sâu hơn, đóng gói thành một thư viện Micro-DeepLearning với các API dễ gọi (tương tự như `torch.nn.Sequential` của PyTorch).

— HẾT —